DEPARTMENT OF ECE, UCLA
ECE 149: THE FOUNDATIONS OF COMPUTER VISION

INSTRUCTOR: Prof. Achuta Kadambi
TA: Rishi Upadhyay

NAME: Brandon Cheung

HOMEWORK 4

PROBLEM	TYPE	TOPIC	MAX. POINTS
1	Analytical	Machine Learning Basics	10
2	Coding	Training a Classifier	15
3	Interview Questions	Miscellaneous	15
4	Analytical (Bonus)	Generative adversarial networks	5

Motivation

The problem set gives you a basic exposure to machine learning approaches and techniques used for computer vision tasks such as image classification. You will train a simple classifier network on CIFAR-10 dataset using google colab. We have provided pytorch code for the classification question, you are free to use any other framework if that's more comfortable.

The problem set consists of two types of problems:

- analytical questions to solidify the concepts covered in the class, and
- coding questions to provide a basic exposure to building a machine learning classifier using pytorch.

This problem set also exposes you to a variety of machine learning questions commonly asked in job/internship interviews

Homework Layout

The homework consists of 3 problems in total, with subparts for each problem. There are 2 types of problems in this homework - analytical and coding. All the problems need to be answered in the Overleaf document. Make a copy of the Overleaf project from here `https://www.overleaf.`

com/read/drgztfysthzv#15de92, and fill in your answers for the questions in the solution boxes provided.

For the analytical questions you will be directly writing their answers in the space provided below the questions. For the coding problems you need to use the Jupyter notebooks from here: https://colab.research.google.com/drive/1ciRDVvVRyIx5c48yF21BWf-ttnOtx83k?usp=sharing(see the Jupyter notebook for each sub-part which involves coding). You are provided with 1 jupyter notebook for Problem 2. After writing your code in the Jupyter notebook you need to copy paste the same code in the space provided below that question on Overleaf. In some questions you are also required to copy the saved images (from Jupyter) into the solution boxes in Overleaf. For the classification question, upload the provided notebook to google colab, and change the runtime type to GPU for training the classifier on GPU. Refer to question 2 for more instructions/details.

Submission

Submission will be done via Gradescope. You will need to submit three things: (1) this PDF with answers filled out (from Overleaf), (2) Your .ipynb file, (3) a PDF printout of your .ipynb file with all cells executed. You do not need to create a folder, you will be able to upload all three files directly to gradescope.

Software Installation

You will need Jupyter to solve the homework. You may find these links helpful:

- Jupyter (https://jupyter.org/install)
- Anaconda (https://docs.anaconda.com/anaconda/install/)

1 Machine Learning Basics (10 points)

1.1 Calculating gradients (2.0 points)

A major aspect of neural network training is identifying optimal values for all the network parameters (weights and biases). Computing gradients of the loss function w.r.t these parameters is an essential operation in this regard (gradient descent). For some parameter w (a scalar weight at some layer of the network), and for a loss function L, the weight update is given by $w := w - \alpha \frac{\partial L}{\partial w}$, where α is the learning rate/step size.

Consider (a) w, a scalar, (b) $\mathbf{x}$, a vector of size $(m \times 1)$, (c) $\mathbf{y}$, a vector of size $(n \times 1)$ and (d) $\mathbf{A}$, a matrix of size $(m \times n)$. Find the following gradients, and express them in the simplest possible form (boldface lowercase letters represent vectors, boldface uppercase letters represent matrices, plain lowercase letters represent scalars):

- $z = \mathbf{x}^T\mathbf{x}$, find $\frac{dz}{d\mathbf{x}}$
- $z = Trace(\mathbf{A}^T\mathbf{A})$, find $\frac{dz}{d\mathbf{A}}$
- $z = \mathbf{x}^T\mathbf{A}\mathbf{y}$, find $\frac{\partial z}{\partial \mathbf{y}}$
- $\mathbf{z} = \mathbf{A}\mathbf{y}$, find $\frac{d\mathbf{z}}{d\mathbf{y}}$

You may use the following formulae for reference:

$$\frac{\partial z}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial z}{\partial x_1} \\ \frac{\partial z}{\partial x_2} \\ \vdots \\ \frac{\partial z}{\partial x_m} \end{bmatrix}, \frac{\partial z}{\partial \mathbf{A}} = \begin{bmatrix} \frac{\partial z}{\partial A_{11}} & \frac{\partial z}{\partial A_{12}} & \cdots & \frac{\partial z}{\partial A_{1n}} \\ \frac{\partial z}{\partial A_{21}} & \frac{\partial z}{\partial A_{22}} & \cdots & \frac{\partial z}{\partial A_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z}{\partial A_{m1}} & \frac{\partial z}{\partial A_{m2}} & \cdots & \frac{\partial z}{\partial A_{mn}} \end{bmatrix}, \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & \cdots & \frac{\partial y_n}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_n}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_m} & \frac{\partial y_2}{\partial x_m} & \cdots & \frac{\partial y_n}{\partial x_m} \end{bmatrix}$$

$$\boxed{\frac{dz}{d\mathbf{x}} = 2\mathbf{x}, \frac{dz}{d\mathbf{A}} = \mathbf{A}, \frac{\partial z}{\partial \mathbf{y}} = \mathbf{A}^T\mathbf{x}, \frac{d\mathbf{z}}{d\mathbf{y}} = \mathbf{A}^T}$$

1.2 Deriving Cross entropy Loss (6.0 points)

In this problem, we derive the cross entropy loss for binary classification tasks. Let $\hat{y}$ be the output of a classifier for a given input x. y denotes the true label (0 or 1) for the input x. Since y has only 2 possible values, we can assume it follow a Bernoulli distribution w.r.t the input x. We hence wish to come up with a loss function $L(y, \hat{y})$, which we would like to minimize so that the difference between $\hat{y}$ and y reduces. A Bernoulli random variable (refresh your pre-test material) takes a value of 1 with a probability k, and 0 with a probability of $1 - k$.

(i) Write an expression for $p(y|x)$, which is the probability that the classifier produces an observation $\hat{y}$ for a given input. Your answer would be in terms of $y, \hat{y}$. Justify your answer briefly.

$p(y|x) = \hat{y}^{(y)}(1-\hat{y})^{1-y}$, where $\hat{y} = k$

(ii) Using (i), write an expression for $\log p(y|x)$. $\log p(y|x)$ denotes the log-likelihood, which should be maximized.

$\log p(y|x) = y \log \hat{y} + (1-y)\log(1-\hat{y})$

(iii) How do we obtain $L(y,\hat{y})$ from $\log p(y|x)$? Note that $L(y,\hat{y})$ is to be minimized

$L(y,\hat{y}) = \frac{1}{N}\sum_i -[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$, where i is the ith sample out of N total samples

1.3 Perfect Classifier (?) (2.0 points)

You train a classifier on a training set, achieving an impressive accuracy of 100 %. However to your disappointment, you obtain a test set accuracy of 20 %. For each suggestion below, explain why (or why not) if these suggestions may help improve the testing accuracy.

1. Use more training data
2. Add L2 regularization to your model
3. Increase your model size, i.e. increase the number of parameters in your model
4. Create a validation set by partitioning your training data. Use the model with highest accuracy on the validation set, not the training set.

1. Yes, increasing training data forces the model to generalize as it is unable to overfit all samples.
2. Yes, regularization reduces the model complexity and minimizes the square of the magnitude of the weights.
3. No, this increases the complexity of the model which is for underfitting instead.
4. Yes, this chooses the best model with best generalization ability.

2 Implementing an image classifier using PyTorch (15.0 points)

In this problem you will implement a CNN based image classifier in pytorch. We will work with the CIFAR-10 dataset. Follow the instructions in jupyter-notebook to complete the missing parts. For this part, you will use the notebook named PSET4_Classification. For training the model on colab gpus, upload the notebook on google colab, and change the runtime type to GPU.

2.1 Loading Data (2.0 points)

(i) Explain the function of transforms.Normalize() function (See the Jupyter notebook Q1 cell). How will you modify the arguments of this function for gray scale images instead of RGB images. (ii) Write the code snippets to print the number of training and test samples loaded.

Make sure that your answer is within the bounding box.

i. Images in the range [-1, 1] will be normalized while grayscale images are changed to 0.5.
ii.

```
print(f"Number of training samples: {len(train_data)}")
print(f"Number of test samples: {len(test_data)}")
```

2.2 Classifier Architecture (6.0 points)

(See the Jupyter Notebook) Please go through the supplied code that defines the architecture (cell Q2 in the Jupyter Notebook), and answer the following questions.

1. Describe the entire architecture. The description for each layer should include details about kernel sizes, number of channels, activation functions and the type of the layer.
2. What does the padding parameter control?
3. Briefly explain the max pool layer.
4. What would happen if you change the kernel size to 3 for the CNN layers without changing anything else? Are you able to pass a test input through the network and get back an output of the same size? Why/why not? If not, what would you have to change to make it work?
5. While backpropagating through this network, for which layer you don't need to compute any additional gradients? Explain Briefly Why.

1. Layer 1: convolution layer, 3 input channels, 6 output channels, kernel size 5. Layer 2: ReLU layer, activation function ReLU. Layer 3: pooling layer with max pooling, kernel size 2. Layer 4: convolution layer, 6 input channels, 16 output channels, kernel size 5. Layer 5: ReLU layer, activation function ReLU. Layer 6: pooling layer with max pooling, kernel size 2. Layer 7: fully connected layer, input size is 400, output size 120. Layer 8: ReLU layer, activation function ReLU. Layer 9: fully connected layer, input size 120, output size 84. Layer 10: ReLU layer, activation function ReLU. Layer 11: fully connected layer, input size is 84,

output size 10.
2. It controls the additional size of zero padding.
3. It does the downsampling.
4. No, a test input cannot be passed through the network and get back an output of the same size since the output will shrink due to the receptive field size shrinking without adding padding. Change the flatten dimension before fully connected layers and the input size of the first fully connected layer.
5. ReLU layers, based on ReLU, the gradient of non-negative number elements are 1 while gradients of negative number elements are 0.

2.3 Training the network (3.0 points)

(i) (See the Jupyter notebook.) Complete the code in the jupyter notebook for training the network on a CPU, and paste the code in the notebook. Train your network for 3 epochs. Plot the running loss (in the notebook) w.r.t epochs.

```
### Complete the code in the training box

## for reproducibility
torch.manual_seed(7)
np.random.seed(7)

## Instantiating classifier
device = torch.device("cpu")
net = Net()
net.to(device)

## Defining optimizer and loss function
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)


## Defining Training Parameters

num_epochs = 3
running_loss_list = [] # list to store running loss in the code below
for epoch in range(num_epochs):  # loop over the dataset multiple times
    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
```

```
        #==================================================#
        # Fill in the training loop here.
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        #==================================================#
        # print statistics
        running_loss += loss.cpu().item()
        if i % 250 == 249:    # print every 250 mini-batches
            print('[{}, {}] loss: {:.3f}'.format(epoch + 1, i + 1,
             ↪  running_loss / 250))
            running_loss_list.append(running_loss)
            running_loss = 0.0

print('Training Complete')
PATH = './net.pth'
torch.save(net.state_dict(), PATH)

## complete the code to plot the running loss per 250 mini batches
 ↪  curve

def plot_loss_curve(running_loss_list):
  plt.plot(running_loss_list)
plot_loss_curve(running_loss_list)
```

(ii) (See the Jupyter notebook.) Modify your training code, to train the network on the GPU. Paste here the lines that need to be modified to train the network on google colab GPUs. Train the network for 20 epochs

```
### Complete the code in the training box

## for reproducibility
torch.manual_seed(7)
np.random.seed(7)

## Instantiating classifier
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
net = Net()
```

```python
net.to(device)

## Defining optimizer and loss function
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)


## Defining Training Parameters

num_epochs = 20
running_loss_list = [] # list to store running loss in the code below
for epoch in range(num_epochs):  # loop over the dataset multiple times
    running_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
        #================================================#
        # Fill in the training loop here.
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        #================================================#
        # print statistics
        running_loss += loss.cpu().item()
        if i % 250 == 249:    # print every 250 mini-batches
            print('[{}, {}] loss: {:.3f}'.format(epoch + 1, i + 1,
             ↪  running_loss / 250))
            running_loss_list.append(running_loss)
            running_loss = 0.0

print('Training Complete')
PATH = './net.pth'
torch.save(net.state_dict(), PATH)

## complete the code to plot the running loss per 250 mini batches
 ↪  curve

def plot_loss_curve(running_loss_list):
  plt.plot(running_loss_list)
plot_loss_curve(running_loss_list)
```

(iii) Explain why you need to reset the parameter gradients for each pass of the network

If parameter gradients are not reset for each pass of network, the previous gradients will accumulate but are not helpful for updating network weights.

2.4 Testing the network (4.0 points)

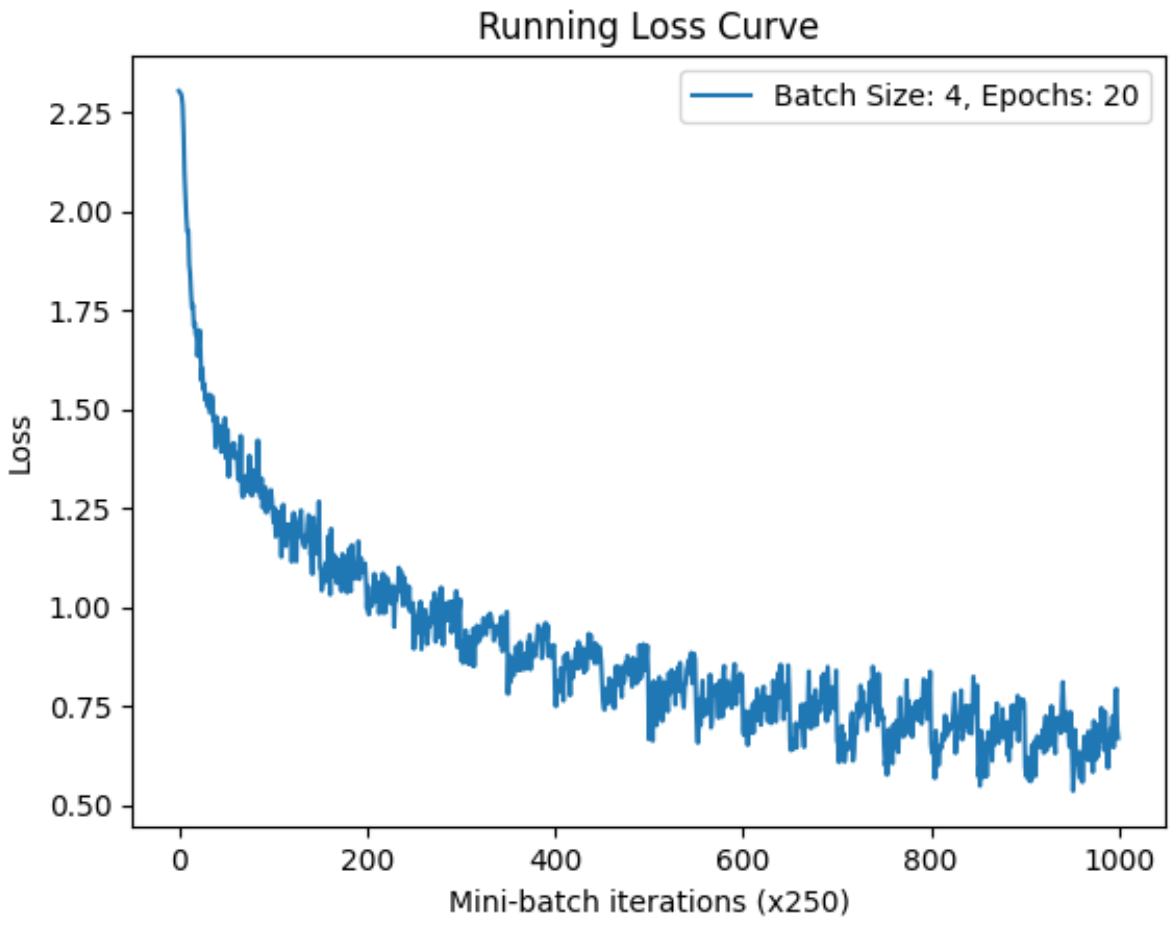
(i) (See the jupyter-notebook) Complete the code in the jupyter-notebook to test the accuracy of the network on the entire test set.

```
### Accuracy on whole data set
correct = 0
total = 0
with torch.no_grad():
    for data in test_loader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
acc  = correct/total*100
print('Accuracy of the network on the 10000 test images: %d %%' % (acc))
```

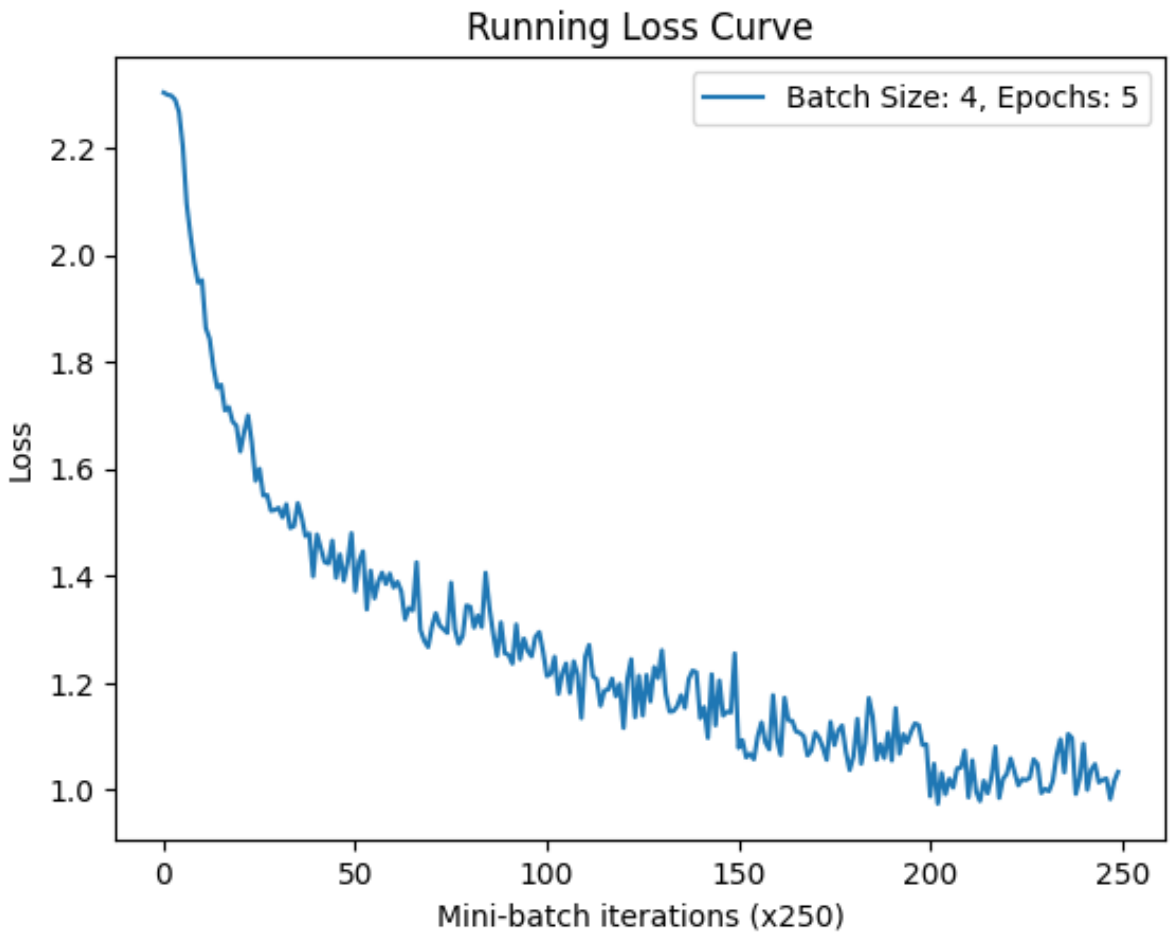
(ii) Train the network on the GPU with the following configurations, and report the testing accuracies and running loss curves -

- Training Batch Size 4, 20 training epochs
- Training Batch Size 4, 5 epochs
- Training Batch Size 16, 5 epochs
- Training Batch Size 16, 20 epochs

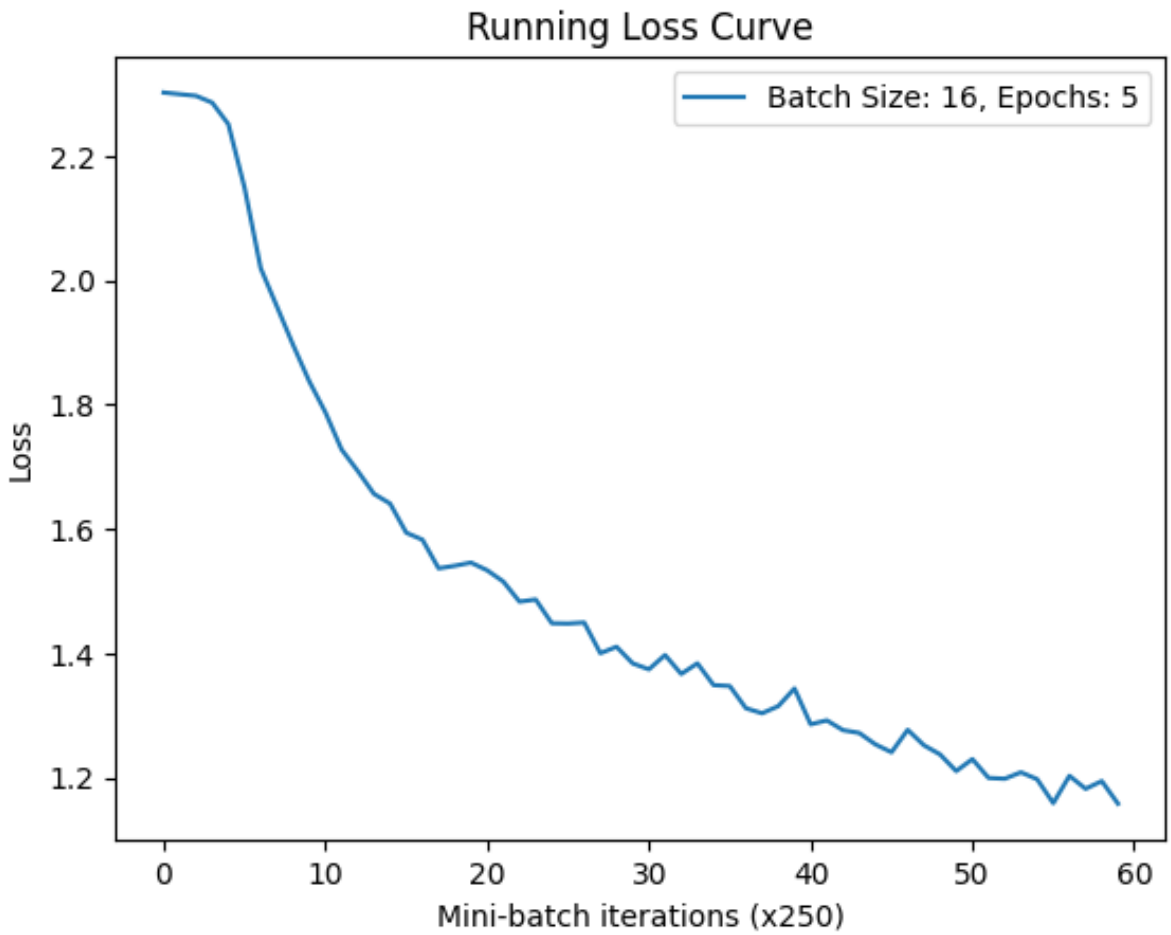
Training Batch Size 4, 20 training epochs: accuracy 61%



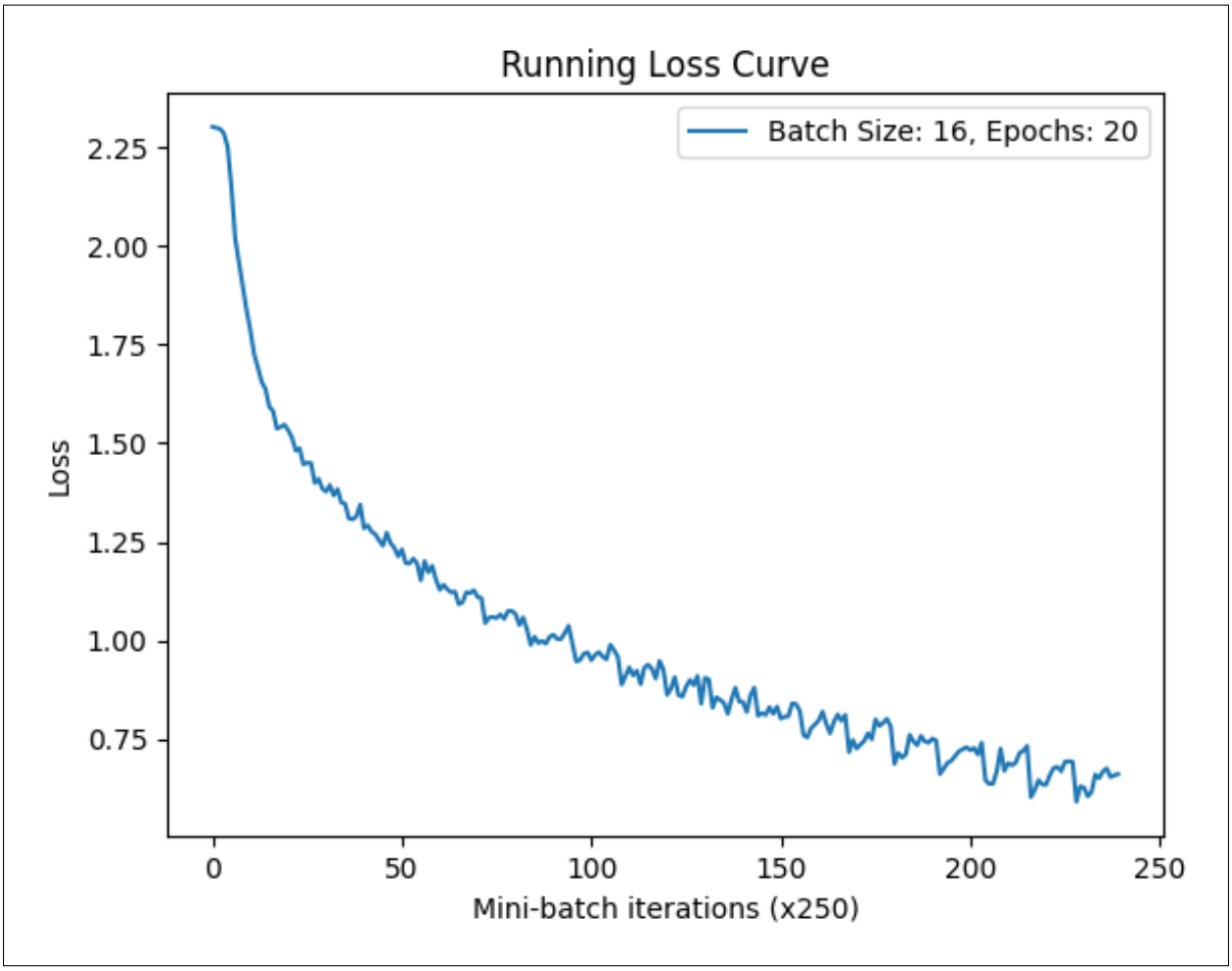
Training Batch Size 4, 5 training epochs: accuracy 59%



Training Batch Size 16, 5 training epochs: accuracy 58%



Training Batch Size 16, 20 training epochs: accuracy 65%



(iii) Explain your observations in (ii)

Accuracy, batch size, and epochs are not strictly positively correlated. Batch sizes should not be too small or too big. Epochs should not be too big to avoid overfitting.

3 Interview Questions (15 points)

3.1 Batch Normalization (4 points)

Explain
(i) Why batch normalization acts as a regularizer.
(ii) Difference in using batch normalization at training vs inference (testing) time.

i. Every mini-batch is scaled with its own mean and standard deviation which introduces noise to each layer, acting as a regularizer.

ii. Batch layers see the whole batch to computer the mean and variance while also learning their parameter during training time. At test time, the layer is only supposed to see one test data point while using the estimated mean and variance captured during training.

3.2 CNN filter sizes (4 points)

Assume a convolution layer in a CNN with parameters $C_{in} = 32$, $C_{out} = 64, k = 3$. If the input to this layer has the parameters $C = 32, H = 64, W = 64$.

(i) What will be the size of the output of this layer, if there is no padding, and stride = 1
(ii) What should be the padding and stride for the output size to be $C = 64, H = 32, W = 32$

i. $C = 64, H = 62, W = 62$
ii. $S = 3, P = 16$

3.3 L2 regularization and Weight Decay (4 points)

Assume a loss function of the form $L(y, \hat{y})$ where y is the ground truth and $\hat{y} = f(x, w)$. x denotes the input to a neural network (or any differentiable function) $f()$ with paramters/weights denoted by w. Adding $L2$ regularization to $L(y, \hat{y})$ we get a new loss function $L'(y, \hat{y}) = L(y, \hat{y}) + \lambda w^T w$, where λ is a hyperparameter. Briefly explain why $L2$ regularization causes weight decay. Hint: Compare the gradient descent updates to w for $L(y, \hat{y})$ and $L'(y, \hat{y})$. Your answer should fit in the given solution box.

Minimizing the loss function with $L2$ regularization minimizes the weight of our neural network. This new loss function guarantees that the weight is kept as small as possible while also minimizing the error between outputs and true labels.

3.4 Why CNNs? (3 points)

Give 2 reasons why using CNNs is better than using fully connected networks for image data.

CNNs are better than fully connected networks for image data due to computational tractability and explicit hierarchical representation of features.

4 GAN (Bonus) (5.0 points)

4.1 Understanding GANs- Loss function (2.0 points)

Mathematically express the overall GAN loss function being used. For a (theoretically) optimally trained GAN: (a) what is the ideal behavior of the discriminator, and (b) what is the value of the overall loss function?

GAN loss function: $\min_G \max_D V(D,G) = \mathbf{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbf{E}_{z\sim p_z(z)}[\log(1-D(G(z)))]$, where $x \sim p_{data}(x)$ is real data and $z \sim p_z(z)$ is fake data
a) The ideal behavior of the discriminator is $D(x) = D(G(z)) = 0.5$ at optimal convergence
b) $-2\log(2)$

4.2 Understanding GANs- Gradients (2.0 points)

Assume that you are working with a GAN having the following architecture:

Generator: Input: $\mathbf{x}$ shape $(2,1) \rightarrow$ Layer: $\mathbf{W_g}$ shape $(5,2) \rightarrow$ ReLU $\rightarrow$ Output: $\mathbf{y}$ shape $(5,1)$
Discriminator: Input: $\mathbf{z}$ shape $(5,1) \rightarrow$ Layer: $\mathbf{W_d}$ shape $(1,5) \rightarrow$ Sigmoid $\rightarrow$ Output: b shape $(1,1)$

Therefore, the generator output is given by, $\mathbf{y} = ReLU(\mathbf{W_g x})$, and the discriminator output is given by $b = Sigmoid(\mathbf{W_d z})$. Express the gradient of the GAN loss function, with respect to the weight matrices for the generator and discriminator.

You may use the following information:

$$Sigmoid(x) = \frac{1}{1+e^{-x}}$$

$$ReLU(x) = \begin{cases} x, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$

Hint: Remember that the input here is a vector, not an image.

$\frac{\partial L_D}{\partial \mathbf{W_d}} = -(1-D(x))\mathbf{z^T} + D(G(z))\mathbf{z^T}$, $\frac{\partial L_G}{\partial \mathbf{W_g}} = -(1-D(G(z)))\mathbf{W_d}^{\mathbf{T}} ReLU'(\mathbf{W_g x})\mathbf{x^T}$,
where $ReLU'(x) = 1$ if $x \geq 0$, $ReLU'(x) = 1$ if $x < 0$

4.3 Understanding GANs- Input distributions (1.0 points)

While training the GAN, the input is drawn from a normal distribution. Suppose in a hypothetical setting, each time the input is chosen from a different, randomly chosen probability distribution. How would this affect the training of the GAN? Justify your answer mathematically.

Since p_z changes at each iteration, generator gradients will depend on p_{z_i} which increases instability in training: $\frac{\partial L_{G_i}}{\partial G_i} = -\mathbf{E}_{z \sim p_{z_i}}[\frac{\partial \log(1-D(G_i(z)))}{\partial G_i}]$, discriminator gradients will also vary inconsistently which causes it to fail: $\frac{\partial L_D}{\partial \mathbf{W_d}} = -(1-D(G_i(z)))\mathbf{z^T}$